HELLENIC REPUBLIC
National and Kapodistrian University of Athens
EST. 1837

DEPARTMENT OF
INFORMATICS & TELECOMMUNICATIONS

Artificial Intelligence II
Homework 1
Vaccine Sentiment Classifier

Anna-Aikaterini Kavvada - cs2210009

November 25, 2021

Professor: Manolis Koumbarakis
Class: YS19 - Fall 2021

1 Linear algebra and calculus warm up...

$$\nabla_w MSE(w) = \begin{pmatrix} \frac{\partial}{\partial w_1} MSE(w) \\ \frac{\partial}{\partial w_2} MSE(w) \\ \dots \\ \frac{\partial}{\partial w_n} MSE(w) \end{pmatrix} =$$

$$= \begin{pmatrix} \frac{2}{m} \cdot \sum_{i=1}^{m} (w \cdot x^i - y^i) x_1^{(i)}) \\ \frac{2}{m} \cdot \sum_{i=1}^{m} (w \cdot x^i - y^i) x_2^{(i)}) \\ \dots \\ \frac{2}{m} \cdot \sum_{i=1}^{m} (w \cdot x^i - y^i) x_n^{(i)}) \end{pmatrix} =$$

$$= \frac{2}{m} \cdot \sum_{i=1}^{m} (w \cdot x^i - y^i) \cdot \begin{pmatrix} \sum_{i=1}^{m} x_1^{(i)} \\ \sum_{i=1}^{m} x_2^{(i)} \\ \dots \\ \sum_{i=1}^{m} x_n^{(i)} \end{pmatrix}$$

Observations:

- We can observe that the above array is the array X^T
- The outer sum can also be expressed in matrix notation as $X_w - Y$
- We also know that for the inner product we have $a \cdot b = b \cdot a$

From the above we reach the following:

$$\nabla_w MSE = \frac{2}{m} \cdot X^T \cdot (X_w - Y) \quad \square$$

2 Vaccine Sentiment Classifier

In this exercise we have to develop a vaccine sentiment classifier using softmax regression. The classifier will be trained on twitter data and will distinguish 3 classes (neutral, anti-vax and pro-vax). The classifier is trained with the datasets provided on eclass.

To begin with, we first read the train and datasets and store them as dataframes. Following is a visual representation of the classes distribution in the training dataset.

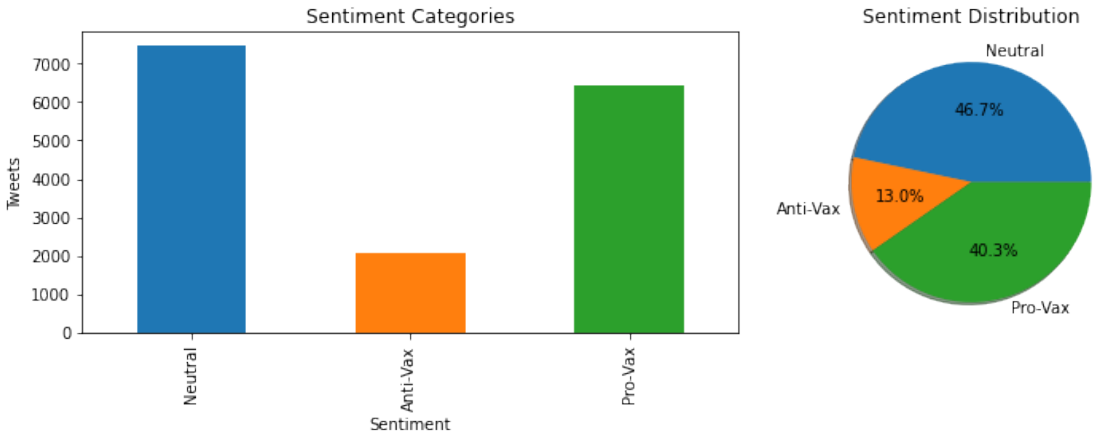

Figure 1:

Heuristic Features

Before the cleaning process of the data, we proceed to some basic feature extraction to augment the information available to our model during its training cycle. The features included are the ones below:

- **len:** Length of tweet
- **n_words** Number of words in tweet
- **n_stopwords** Number of stopwords in tweet

Feature extraction is applied on both train and test datasets. After this process we have augmented our dataframes.

Data Preproces

Text Data Cleaning

For the data cleaning process, the following were included:

- Remove all whitespaces and newlines
- Remove all special characters and punctuation
- Replace contractions with full words (despite that they might be removed later from the stopwords)
- Splitted attached words e.g. “ForTheWin” =¿ “For The Win”
- Lowered all capital letters
- Removed numbers from corpus
- Standarizing sentences

Both train and test data are preprocessed in the same exact way.
reference link: https://www.geeksforgeeks.org/python-efficient-text-data-cleaning/

Stemming

To enhance the classification performance we proceeded to stem the data of Title Content column.
Stemming is a Text Normalization (or sometimes called Word Normalization) techniques in the field of Natural Language Processing that are used to prepare text, words, and documents for further processing. Stemming helps us to achieve the root forms (sometimes called synonyms in search context) of inflected (derived) words. Thus, we define as Stem (root) the part of the word to which we add inflectional (changing/deriving) affixes such as (-ed,-ize, -s,-de,mis). So stemming a word or sentence may result in words that are not actual words. Stems are created by removing the suffixes or prefixes used with a word.
For this process we used PorterStemmer for the English language.
Stemming is applied both on the train and test dataset.

Standardization

Standardization of a dataset is a common requirement for many machine learning estimators: they might behave badly if the individual features do not more or less look like standard normally distributed data (e.g. Gaussian with 0 mean and unit variance).
For instance many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the L1 and L2 regularizers of linear models) assume that all features are centered around 0 and have variance in the same order. If a feature has a variance that is orders of magnitude larger that others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.
All the features extracted earlier from the dataset corpus, do not necessarily follow the normal distribution curve, which is proved to enhance the algorithms learning process, as explained in the previous paragraphs.

Bag of Words

The creation of the BoW is approached in two ways. One is with the use of the TF-IDF vectorizer and the other with the Count Vectorizer, which is a simpler approach.
We use the tfidf vectorizer to create the bag of words. TF-IDF stands for Term Frequency-Inverse Document Frequency. The Bag of Words (BoW) model is the simplest form of text representation in numbers. Like the term itself, we can represent a sentence as a bag of words vector (a string of numbers).
A simple Bag of Words, like one created with the use of CountVectorizer class, just creates a set of vectors containing the count of word occurrences in the corpus given. On the other hand, the TF-IDF model contains information on the more important words and the less important ones as well. In this case we create a BoW with the default number of features.

Classifier

In general for each classification algorithm, the process we follow is the same:

1. Create the bag of words for the tweets.
2. Stack the aforementioned BoWs along with the extracted features from the train or test data respectively, in order to create a single array containing all the information we have available.
3. Initialize the classifier of our choice with its specific parameters. The latter were decided after multiple experiments.
4. Fit train data in the model.
5. Make predictions for the test data.

The above process is repeated k times, each time on a different portion of the data. In every loop this chunk is augmented and always arbitrary, except for the last loop, where the whole dataset is given to the model for trainning. In this experiment k=5.
The process described in the bullets is repeated twice inside every loop, one time for the BoW created with the TF-IDF vectorizer and one for the Count Vectorizer.

Logistic Regression

Logistic regression is common and is a useful regression method for solving the binary classification problems, not like the problem at hand, where we have three different problem classes. It is one of the most simple and commonly used Machine Learning algorithms for two-class classification. It is easy to implement and can be used as the baseline for any binary classification problem. Its basic fundamental concepts are also constructive in deep learning. Logistic regression describes and estimates the relationship between one dependent binary variable and independent variables. Some of the properties of Logistic regression are:

- The dependent variable in logistic regression follows Bernoulli Distribution.
- Estimation is done through maximum likelihood.

- No R Square, Model fitness is calculated through Concordance, KS-Statistics.

Learning Curves

Below we can see the learning curves for each model:

Logistic Regression with TF-IDF

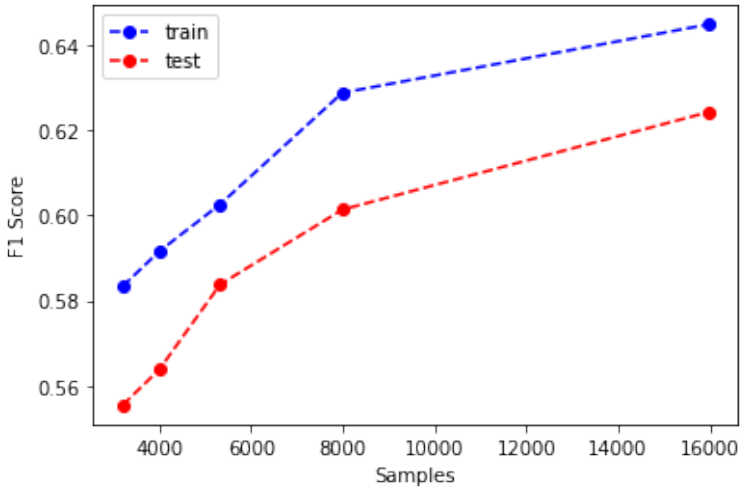

Figure 2: Learing Curve for Logistic Regression with TF-IDF

	precision	recall	f1-score	support
0	0.72	0.80	0.76	1065
1	0.64	0.02	0.05	296
2	0.60	0.72	0.66	921
accuracy			0.66	2282
macro avg	0.65	0.51	0.49	2282
weighted avg	0.66	0.66	0.62	2282

Figure 3: Test Set Classification Report for Logistic Regression with TF-IDF

We see the two curves getting closer to each other while the number of samples from the train dataset increases.

Logistic Regression with Count Vectorizer

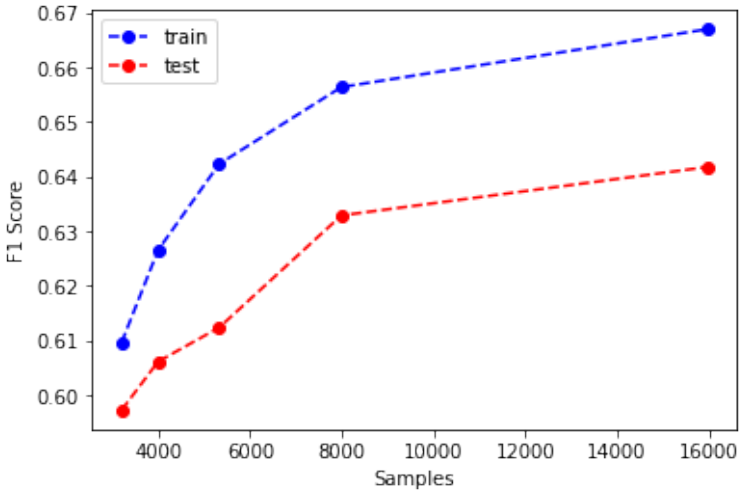

Figure 4: Learing Curve for Logistic Regression with Count Vectorizer

	precision	recall	f1-score	support
0	0.74	0.80	0.76	1065
1	0.68	0.07	0.13	296
2	0.61	0.73	0.66	921
accuracy			0.67	2282
macro avg	0.67	0.53	0.52	2282
weighted avg	0.68	0.67	0.64	2282

Figure 5: Test Set Classification Report for Logistic Regression with Count Vectorizer

We see the two curves getting closer to each other while the number of samples from the train dataset increases.

Model Comparison

It is evident that the two models are quite close regarding their performance, while trying to solve the same problem. It should be noted that in both experiments, the parameter tuning of the models used, was decided after

extensive testing with multiple different combinations. In both cases the curves appear to have the behavior we want in order to avoid either ending up with an overfitted model or an underfitted one. However, it is quite clear that this classification algorithm is not the best option. Logistic regression seems to be inappropriate for non-binary classification problems as this one.